

Table of Contents

Claude Code Orchestration — Requirements Specification

Version 3.2.1 · Updated 2026-05-24 · Author: Valverde · Supersedes “Claude Code Improvements.pdf” + v1, v2, v3.0, v3.1, v3.2.0 of this spec

v3.2.1 changes summary (multi-platform setup + update ergonomics): - **Bootstrap** — Cross-platform Python 3 detection. The script now tries python3 first, falls back to python (verifying it's Python 3.x), and on Windows Git Bash auto-creates a python3.exe shim alongside python.exe when only python is available. Fails fast with a platform-specific install command if no Python 3 is found. - **Bootstrap** — <github-owner> placeholder replaced with valverdesolera in --print-plugins output and the “Next steps” message, now that the repo is public. - **Bootstrap** — Final-message tip pointing users at the new /cco-update slash-command for easier updates. - **New custom slash-command /cco-update** — Wraps /plugin marketplace update cc-orchestration and /plugin update claude-code-orchestration@cc-orchestration into one step, with fallback to uninstall+reinstall on older Claude Code versions. Lives at plugins/claude-code-orchestration/commands/cco-update.md. - **CI: GitHub Actions auto-tag workflow** (.github/workflows/auto-tag.yml) watches plugin.json and marketplace.json on push to main; verifies both versions agree; auto-creates and pushes the matching vX.Y.Z tag. Idempotent. Eliminates manual git tag + git push step. - **Repo went public**. Eliminated the entire auth flow (SSH keys, PATs, collaborator invites) for cloning, installing, and updating across machines. - **README.md rewritten** as a 30-second-install landing page with current layout (no more outdated marketplace/ path; reflects restructured root-level .claude-plugin/). - **INSTALL.md rewritten** around the simple case first, with per-OS notes (macOS / Linux / Windows-Git-Bash) and a focused troubleshooting section covering: pasted-too-many-plugin-installs, stale claude npm install, macOS Homebrew startup error, zsh bad pattern: #, Windows WSL routing in PowerShell, Windows Python 3 detection, OneDrive lock files, fresh-machine auth, and false-positive comment-policy bypasses.

v3.2 changes summary (parity pass against the user's global engineering rules): - **\$3** — Added verification-status discipline: unverified external behavior must be labeled UNVERIFIED in pre- and post-implementation responses; not hidden, not a hard stop. - **\$4** — Added “prefer reusing existing utilities and abstractions” as an explicit positive directive in codebase research. - **\$5 step 8** — Lint and type-check are now first-class loop gates alongside tests; the agent reports which gates ran and which were not configured. - **\$11** — Post-implementation response now requires: “Summary of Documentation Changes” (rationale, not just file list), “Tests / Lint / Type-Check Results” (replaces the generic “Validation Run”), “Rule Compliance Self-Check” (note any rule whose compliance is uncertain or partial), and explicit inclusion of UNVERIFIED claims in Remaining Risks. - **\$13** — Added “prefer the simplest solution that matches confirmed requirements and

current architecture” as a positive directive; complexity must be justified by a confirmed requirement, not anticipated future needs. - **\$15** — Expanded “Canonical per-dependency documentation” subsection: <CanonicalName>Documentation.md naming, one-file-per-dependency, check-before-create, required-contents schema, on-change and on-removal rules. Pulled out of the documentation-refresh skill into the narrative authority so every agent sees it on every session.

v3.1 changes summary (preserved): - Bootstrap added `clean_stale_personal_config()` to back up `~/.claude.json` and remove the `microsoftLearn MCP` entry on first run per machine. - Restructured repo for GitHub-direct install (`.claude-plugin/marketplace.json` at root; `plugins/claude-code-orchestration/...` underneath; `bootstrap/`, `docs/` siblings). - Added meta-architecture-reviewer agent (audits the plugin itself).

v3 changes summary (in response to the self-audit): - Added meta-architecture-reviewer agent (the missing requirement A5 from the third message). Read-only; audits the plugin itself. - Established `reference/recommended-plugins.json` as SINGLE SOURCE OF TRUTH for the 15-plugin list. `CLAUDE.md`, `INSTALL`, `bootstrap`, and `plugin.json` all reference it. No more drift across files. - Consolidated the review pipeline to ONE policy reviewer + ONE bug-finder pass per unit (`CLAUDE.md` \$5). No more 5-reviewer pile-up. - Added graceful degradation (`CLAUDE.md` \$20.5). Each agent now has documented fallback behavior when an official plugin isn’t installed. - Made `Limitations.md` + `EdgeCases.md` HARD GATES in `implementation-planner`. Cannot produce `Plan-Final.md` without 3+ limitations entries and human-confirmed edge cases per stage. - Pre-implementation response template (`CLAUDE.md` \$10) now includes Edge Cases and Limitations sections; orchestrator must wait for explicit confirmation of all three (Assumptions + Edge Cases + Parallelization). - Added Postman MCP section (`CLAUDE.md` \$20.6) — explicit install/use guidance. - Documented the workbook-path decision (`CLAUDE.md` \$20.7) — kept `docs/ignored/workbooks/<slug>/` over original `docs/temp/<team>/` with rationale.

v2 changes summary (preserved): - \$0 Authority: official plugins are tools, never replacements for custom agents - \$4 Agent inventory: +4 agents (architecture-enforcer, data-architect, comment-policy-checker, parallel-research-coordinator) - \$5 Skill inventory: +9 skills - \$6 New workflows: greenfield-vs-brownfield, plan-review-cycle, parallel-codebase-research-cycle, parallelization-decision, pr-merge-conflict-wait, assumption-validation-tests - \$9 MCP routing reframed: vendor-specific source FIRST, context7 generic fallback - \$10 New guardrails: SG10–SG14 (strengthened comment policy, parallel-decision, PR-merge-wait) - \$15 New: plan folder organization rule - \$16 New: mobile/remote coding section - \$17 New: local git hygiene story (`.git/info/exclude`) - \$18 New: 15 plugin dependencies declared

This document is the source of truth for what the orchestration architecture must do. Each requirement is numbered, has explicit acceptance criteria, and is traceable to the agent / skill / hook that implements it. Open questions are listed in \$13.

1. Goals

- **G1** — Improve the coding cycle by decomposing it into an orchestrated set of specialist subagents instead of one monolithic agent.
- **G2** — Eliminate assumptions: every uncertainty becomes an explicit question to the human reviewer before code is written.
- **G3** — Keep all transient Claude-related artifacts (plans, workbooks, context snapshots) **out of the team repo**.
- **G4** — Guarantee that no commit, push, or PR carries Claude/Anthropic/AI attribution or process metadata (Jira IDs, branch names, plan references) in code comments or commit messages.
- **G5** — Enforce a mandatory **implementation feedback loop** (code → context refresh → review → test → docs) before any unit is considered done.
- **G6** — For bugs, enforce a **root-cause convergence loop** using independent investigations before claiming a root cause.
- **G7** — Distribute the architecture cleanly: one Claude Code plugin install + one small bootstrap script.

2. Non-goals

- **NG1** — This is **not** a team-wide engineering policy enforcement system. It is a personal/private overlay; the team's existing `.claude/`, `CLAUDE.md`, and `.mcp.json` are never touched.
- **NG2** — No agent will deploy, restart services, mutate production data, change feature flags, run database writes/migrations, force-push, rebase, reset, or delete branches without explicit human approval.
- **NG3** — No automated PR creation, commit, or push happens without an explicit human request.
- **NG4** — This architecture does **not** replace existing CI. It complements CI with pre-push validation.
- **NG5** — Agent teams (multi-agent direct communication) are **not** enabled by default; they are an opt-in escalation per §6.7.

3. Glossary

Term	Meaning
Orchestrator	The main subagent (engineering-orchestrator) Claude Code activates by default; coordinates all other subagents.
Subagent	A specialized agent with its own context window; reports back to the orchestrator only.
Agent team	A group of subagents that can message each other directly (Claude Code's agent-teams feature); higher token cost, used only for collaborative reasoning.

Skill	A reusable Markdown procedure under skills/<name>/SKILL.md; auto-invoked by Claude when its description matches the task.
Hook	A shell script triggered by Claude Code at PreToolUse, PostToolUse, etc.; can block or annotate tool calls.
Workbook	A per-task folder under docs/ignored/workbooks/<task>/ with research.md, design.md, handoff.md, notes.md, README.md (timestamped activity log). Gitignored.
Implementation Plan	A long-lived .md under docs/ignored/implementation/ describing staged work, edge cases, and validation. Gitignored.
Codebase Context	A .md snapshot under docs/ignored/context/<module>/ describing a module's purpose, entry points, contracts, constraints. Gitignored.
Module Context File	The per-module document with sections Purpose, Dependencies, Architectural Constraints, Internal Topology, Execution & Data Flow, Cross-Cutting Concerns, Public Interface & Contracts, Directory Layout, Architectural Guardrails. Lives under docs/ignored/context/, never **/.module-context.md in the repo.

4. Agent inventory (functional requirements)

Each row is a hard requirement. The orchestrator's tools: Agent(...) allowlist enumerates exactly this set.

ID	Agent	Function	Read/Write	Implementation
A1	engineering-orchestrator	Decompose work, route to specialists, manage dependencies/parallelism, aggregate results, retry on failure, track state. Highest	R/W (delegates)	agents/engineering-orchestrator.md

		reasoning effort, latest model (Opus, effort: max).		
A2	requirements-product-analyst	Convert request into acceptance criteria, non-goals, edge cases, and explicit questions for the human. No assumptions allowed.	R	agents/requirements-product-analyst.md
A3	external-documentation-researcher	Ground implementations in official docs via Context7 / vendor MCPs / CLIs. Update docs/ignored/<CanonicalName>Documentation.md.	R/W (docs/ignored only)	agents/external-documentation-researcher.md
A4	codebase-researcher	Find relevant files, conventions, dependencies, prior patterns. Parallelizable. Refresh docs/ignored/context/<module>/ snapshots.	R/W (docs/ignored only)	agents/codebase-researcher.md
A5	implementation-planner	Convert requirements into staged plans with assumptions, edge cases, validation, limitations, and loop exit criteria. Each stage waits for	R/W (docs/ignored only)	agents/implementation-planner.md

A6	coding-agent	human confirmation. Implement one approved unit at a time. Returns to feedback loop on failure. No TODOs, placeholders, pseudo-code. Comments never reference plans/tickets/branches/Claude .	R/W (code)	agents/coding-agent.md
A7	code-reviewer	Reviews every changed/added/deleted file against the approved stage, edge cases, architecture, comment policy, and existing patterns. Flags placeholders, drift, over/under-engineering.	R	agents/code-reviewer.md
A8	test-agent	After review passes: run targeted tests first, then broader validation per validation-matrix. Creates regression tests for the bug class addressed. Tests must pass before	R/W (tests)	agents/test-agent.md

		the next stage starts.		
A9	documentation-reviewer	Update docs/ignored/* * after each stage; remove stale/contradictory content. Triggerable manually via run-documentation-review skill.	R/W (docs/ignored only)	agents/documentation-reviewer.md
A10	performance-reviewer	Latency, throughput, scaling, demand, resource usage, and overengineering risk without editing. Cross-checks against the original requirements.	R	agents/performance-reviewer.md
A11	refactor-cleanup-agent	Minimal cleanup for drift, duplication, weak implementation , or unnecessary complexity. Compares actual implementation to the plan.	R/W (code)	agents/refactor-cleanup-agent.md
A12	git-version-control-agent	Read-only git status/diff/branch/commit-prep/merge diagnostics. Knows the repo's	R	agents/git-version-control-agent.md

		branch/PR conventions.		
A13	merge-conflict-resolver	Explicitly approved resolver for conflicted files only.	R/W (specific files only)	agents/merge-conflict-resolver.md
A14	pre-push-guardian	Final gate before push: check staged/outgoing files for forbidden artifacts (docs/ignored/**, env files, .claude/**), validate that tests/builds were run for touched layers, check authorship, block Claude/AI attribution.	R	agents/pre-push-guardian.md
A15	worktree-manager	Plan worktrees, base branches, branch names, .worktreeinclude, env/context copying, and setup steps.	R/W (worktree config)	agents/worktree-manager.md
A16	pr-creator	Inspect repo state, infer PR conventions from .github/pull_request_template.md / CONTRIBUTING.md / recent merged PRs; produce title + body; only creates	R	agents/pr-creator.md

		PR when explicitly requested. Also handles merge-conflict diagnostic. Never includes Claude as co-author.		
A17	pr-reviewer	Reviews PR/branch correctness, risks, tests, docs, conventions, security, performance.	R	agents/pr-reviewer.md
A18	backend-bug-finder	Find backend/API/database/queue/integration/config/security bugs without editing . Outputs strict report format (§7.2). Read-only DB access.	R	agents/backend-bug-finder.md
A19	frontend-bug-finder	Find browser/UI/JS/SSR/layout/all y/performance bugs without editing . Uses Chrome DevTools / Playwright / Storybook MCPs. Outputs strict report format (§7.3).	R	agents/frontend-bug-finder.md

4.1 Hard constraints for all agents

- **HC1** Comments in code **MUST NOT** reference implementation plans, Jira tickets, branches, PRs, Claude, Anthropic, or AI. (Enforced by `post-edit-policy-check.sh` hook and by `code-reviewer`.)
- **HC2** No `.md` document is committed or pushed. (Enforced by Git pre-commit + pre-push hooks installed by the bootstrap.)
- **HC3** No commit/push includes Claude as co-author. (Enforced by Git commit-msg hook.)
- **HC4** No temporary scripts are committed. (Enforced by Git pre-commit hook patterns.)
- **HC5** No agent runs destructive `git/DB/deploy` commands without `ALLOW_*=1` env vars. (Enforced by `guard-git-and-dangerous-bash.sh` hook.)
- **HC6** No agent edits team-visible config: `repo/.claude/**`, `repo/CLAUDE.md`, `repo/mcp.json` — read-only unless explicitly approved. (Enforced by `guard-edit-path-policy.sh` hook.)
- **HC7** No agent mutates external systems via MCP without explicit approval. (Enforced by `guard-mcp-mutations.sh` hook with `CLAUDE_ALLOW_MCP_MUTATION=1`.)
- **HC8** Only the orchestrator may spawn the approved specialist set. Specialists may not spawn further specialists. (Enforced by `guard-agent-spawn.sh` hook.)

5. Skill inventory (procedural requirements)

ID	Skill	Used by	Purpose
S1	implementation-feedback-loop	Orchestrator, coding, review, test, doc agents	The canonical code→context-refresh→review→test→docs loop (§6.1).
S2	implementation-planning	implementation-planner	Requirements → staged plan with assumptions, edge cases, validation, limitations.
S3	codebase-contextualization	Orchestrator, codebase-researcher, coding-agent	Create/refresh docs/ignored/context/<module>/ before coding and after code changes . Parallelizable.
S4	validation-matrix	test-agent	Determine which tests, builds, schema-generation, image builds, lint/type-check, manual checks are required for the

			touched layer (see §6.6).
S5	workbook-management	All	Create docs/ignored/workbooks/<task>/ with timestamped activity log; track agent handoffs.
S6	documentation-refresh	documentation-reviewer	Promote durable workbook facts to docs/ignored/** long-lived docs; remove stale/contradictory content.
S7	safe-git-commit	git-version-control-agent, pre-push-guardian	Pre-commit/pre-push checklist for safe staged files, authorship, attribution.
S8	merge-conflict-handling	merge-conflict-resolver, pr-creator	Safe diagnosis & resolution of merge/rebase/divergent-branch conflicts.
S9	pr-convention-detection	pr-creator	Find PR templates, recent PR patterns, titles, bodies, testing/risk sections.
S10	branch-creation	worktree-manager, git-version-control-agent	Infer branch naming + base branch from conventions; wait for human confirmation.
S11	worktree-handoff	worktree-manager	Choose which docs/context/env/s etup artifacts to copy or regenerate in a worktree.
S12	agent-team-decision	Orchestrator	Decide between subagent, Agent View / background agent, worktree, or

			agent team for the current task (§6.7).
S13	root-cause-convergence	Orchestrator, backend/frontend bug finders	Independent investigations + convergence comparison before claiming root cause (§6.8).
S14	backend-error-triage	backend-bug-finder	Capture / classify / localize / reproduce / report procedure.
S15	observability-correlation	backend-bug-finder	Correlate logs/traces/metrics / deployments/errors across observability tools.
S16	database-debugging	backend-bug-finder	Read-only schema/index/transaction/lock investigation.
S17	backend-security-scan	backend-bug-finder	Backend auth/injection/SSRF / secrets/tenant-isolation review.
S18	frontend-error-triage	frontend-bug-finder	Frontend runtime/hydration/UI/routing/state triage.
S19	browser-reproduction-debugging	frontend-bug-finder	Browser/ DevTools/ Playwright/ console/network/ DOM reproduction.
S20	frontend-observability-correlation	frontend-bug-finder	Frontend production correlation across errors, replays, flags, releases.
S21	design-system-visual-regression	frontend-bug-finder	Storybook/ Chromatic/Figma/ token/layout regression

S22	accessibility-performance-triage	frontend-bug-finder	investigation. A11y, focus, ARIA, contrast, slow rendering, layout shift, bundle, Core Web Vitals.
S23	run-documentation-review	Human (manual)	Manually invoke documentation-reviewer.
S24	run-pre-push-guardian	Human (manual)	Manually invoke pre-push-guardian.
S25	run-pr-review	Human (manual)	Manually invoke pr-reviewer.
S26	run-backend-bug-finder	Human (manual)	Manually invoke backend-bug-finder.
S27	run-frontend-bug-finder	Human (manual)	Manually invoke frontend-bug-finder.

6. Cross-cutting workflows

6.1 Implementation feedback loop (MANDATORY)

For every approved unit:

1. Coding agent implements **only** the approved unit.
2. codebase-contextualization refreshes docs/ignored/context/** for touched modules.
3. code-reviewer reviews every changed/added/deleted file against the approved stage, edge cases, architecture, comment policy, and existing patterns.
4. If review fails → coding fixes minimally → step 2 → step 3.
5. test-agent runs targeted tests first, then broader validation per the validation-matrix skill.
6. If validation fails → step 3 → step 5.
7. documentation-reviewer updates docs/ignored/** and removes stale/contradictory content.
8. Proceed only when review, validation, and docs gates pass — **or the human explicitly accepts the residual risk.**

Acceptance criterion (AC-6.1): the loop is encoded in skills/implementation-feedback-loop/SKILL.md and referenced by CLAUDE.md, engineering-orchestrator, coding-agent, code-reviewer, test-agent, documentation-reviewer, implementation-planner, implementation-planning, and validation-matrix.

6.2 No-assumptions discipline

Before planning or coding, the orchestrator must produce a “pre-implementation response” containing:

- Understanding · Confirmed constraints · Assumptions · Repo findings · Documentation/MCP findings · Relevant existing docs/ignored docs · Docs to create/update · Open questions · Status: waiting for explicit confirmation.

Implementation does not begin until the human explicitly confirms.

AC-6.2: CLAUDE.md §10 (Required pre-implementation response) and requirements-product-analyst enforce this.

6.3 Codebase Contextualization

Before any code is generated on a fresh branch **and** after every code change, a context snapshot is created/refreshed under docs/ignored/context/<module>/<Module>Context.md following the schema in §3 (Glossary → Module Context File).

The MCPs available for this step (used by parallel sub-investigations when budget allows):

- context7 — library docs
- serena — semantic code retrieval/refactoring/debugging
- codegraphcontext — code graph queries
- codanna — function discovery, call relationships
- tree-sitter — AST parsing
- codeql — static analysis
- srclight — code search

AC-6.3: codebase-contextualization skill exists and is referenced by the orchestrator and codebase-researcher. The skill never creates ****/.module-context.md** in the repo — only docs/ignored/context/**.

6.4 Workbook discipline

- Workbooks live under docs/ignored/workbooks/<team>/<task>/.
- Files are timestamped (filename or content header).
- README.md contains a metadata header + an activity log table (Date · Agent · Action).
- Workbooks are **gitignored** (covered by .git/info/exclude patterns installed by the bootstrap).
- Only durable, re-verified facts get promoted from workbook → long-lived docs/ignored/<Name>.md by documentation-refresh.
- Stale workbook content does **not** leak into long-lived docs.

AC-6.4: workbook-management and documentation-refresh skills exist. documentation-reviewer runs after every approved unit.

6.5 Pre-push validation matrix

The validation-matrix skill enumerates which checks are mandatory based on touched paths. Default rules (override per repo):

Touched path	Required check
app/web/src/**	cd app/web && npm run build
src/ephai_intake/** or srv/**	uv run pytest
schemas/queue-messages.json	bash schemas/generate.sh first, then both above
Dockerfile / container build inputs	Build the image, do not rely on npm run dev

These rules are documented in the plugin's CLAUDE.md §6. They are intended as defaults — each consuming repo MUST customize.

AC-6.5: validation-matrix/SKILL.md documents these rules. pre-push-guardian enforces them on outgoing commits.

6.6 Branch + worktree workflow

- Branch name must be inferred from repo conventions (analyze recent branch names, CONTRIBUTING.md, PR title patterns).
- Base branch is the repo's default branch unless the human specifies otherwise.
- Branch creation **waits for human confirmation**.
- Worktree creation copies docs/ignored/ and .env* files per .worktreeinclude, then re-runs the bootstrap so the new tree gets Git hooks and core.hooksPath.
- Worktrees may need their own dependency install / virtualenv / artifact rebuild; the worktree-manager flags this in its handoff.

AC-6.6: branch-creation, worktree-handoff skills + worktree-manager agent + new-branch.sh / new-worktree.sh wrappers exist.

6.7 Subagents vs. agent teams decision

Per Claude Code docs:

	Subagents	Agent teams
Context	Own window; results return to caller	Own window; fully independent
Communication	Report results to main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion + collaboration

Token cost	Lower (summarized back)	Higher (each teammate is a separate Claude instance)
------------	-------------------------	--

Default: subagents (cheaper, predictable). **Escalate to agent teams** only when: (a) two specialists need to compare hypotheses iteratively, (b) the task is collaborative reasoning (e.g. competing root-cause hypotheses), and (c) the human approves the token spend.

AC-6.7: agent-team-decision skill exists and is referenced by the orchestrator.

6.8 Root-cause convergence loop

For high-impact, production-facing, or uncertain bugs:

1. Capture original evidence from issue/logs/trace/CI/browser repro/user steps.
2. Split independent hypotheses by layer when useful.
3. Run independent investigations (parallel finder subagents, or an agent team) — **with explicit human approval**.
4. Compare findings; report convergence or divergence.
5. If findings diverge, do **not** force a conclusion; gather more evidence.
6. After fix, rerun the original reproduction/test/CI/browser/MCP check.

AC-6.8: root-cause-convergence skill exists, referenced by backend/frontend bug finders and the orchestrator.

6.9 Implementation plan reviews

Each implementation plan stage **MUST** include:

- **Assumptions** — explicit, each tagged “assumed” until human-confirmed.
- **Edge cases** — scenarios where the implementation can fall short, produce wrong results, or be wrongfully implemented.
- **Validation** — which tests / builds / manual checks gate this stage.
- **Limitations** — version-specific constraints, framework-specific behaviors, anything the stage will explicitly not cover.
- **Anti-overengineering reminder** — every stage instruction includes “do not over-engineer; if the simplest solution conflicts with the plan, surface it to the human.”

Each plan runs multiple review rounds against the main requirements and against Context7/vendor-docs before implementation begins. Assumptions / edge cases must be human-confirmed.

AC-6.9: implementation-planning skill + implementation-planner agent enforce this.

6.10 Documentation hygiene

- Long-lived docs (docs/ignored/**, not workbooks) must be re-checked after every approved unit for stale/incorrect/missing info.
- Workbooks have timestamps and a progression order; long-lived docs do not — so the relationship is explicit: workbook → documentation-refresh → long-lived doc.

- Stale information in long-lived docs is **rewritten**, not appended to.
- Documentation reviewer is invoked automatically by the orchestrator after every approved unit **and** manually by the human via run-documentation-review skill.

AC-6.10: documentation-refresh skill + documentation-reviewer agent + CLAUDE.md §9 exist and align.

7. Output format contracts

7.1 Pre-implementation response (from §6.2)

```
## Understanding
## Confirmed Constraints
## Assumptions
## Repo Findings
## Documentation/MCP Findings
## Relevant Existing docs/ignored Docs
## Docs To Create / Update
## Open Questions
## Status: waiting for explicit confirmation
```

7.2 Backend bug finder report

```
## Finding Summary
## Evidence Inspected
## Reproduction
## Root Cause (with confidence: high/medium/low)
## Backend Bug Category (API / Auth/authz / Database / Queue/async /
Integration / Config / Performance / Security / CI-test / Unknown)
## Suggested Fix Plan
## Regression Test Recommendation
## Risks / Notes
## Handoff Prompt For Bug Fixer
```

7.3 Frontend bug finder report

```
## Finding Summary
## Evidence Inspected
## Reproduction
## Root Cause (with confidence: high/medium/low)
## Frontend Bug Category (JavaScript/runtime / Component rendering /
SSR-hydration / Routing-navigation / State management / Forms / Network-API /
Auth-session / CSS-layout / Accessibility / Performance / Build-test / Cross-browser-
device / Visual-design regression / Unknown)
## Suggested Fix Plan
## Regression Test Recommendation
## Risks / Notes
## Handoff Prompt For Bug Fixer
```

7.4 PR creator output

```
Observed PR convention: <summary>
## Title: <proposed title>
## Body: <proposed body>
```

Base branch: <base>
Current branch: <branch>
Commands/checks used to inspect changes: <list>
Action: <only on explicit request, `gh pr create --base <base> --head <branch> --title "<title>" --body-file <temp-body-file>` (with --draft when requested)>

8. Distribution & install

Required deliverables: - **D1** A Claude Code plugin (claude-code-orchestration) installable via `/plugin install claude-code-orchestration@<marketplace>` containing agents, skills, hooks, plugin-level `.mcp.json`, plugin-level `settings.json` (default agent), and `CLAUDE.md`. - **D2** A marketplace manifest (`marketplace/.claude-plugin/marketplace.json`) so the plugin is discoverable via `/plugin marketplace add`. - **D3** A repo bootstrap script (`bootstrap/install_repo_bootstrap.sh`) that is idempotent, prompts before changes, and installs only what plugins cannot: Git hooks, `docs/ignored/`, `.git/info/exclude`, `.worktreeinclude`, `core.hooksPath`, `branch/worktree` wrappers. - **D4** An install guide (this document's companion `INSTALL.md` / `.docx`) covering prerequisites, both install paths (Git marketplace, local), daily use, verification, troubleshooting, and uninstall.

AC-8: All four deliverables exist in this package. The plugin frontmatter complies with Claude Code plugin restrictions (no per-agent hooks, `mcpServers`, `permissionMode`). Plugin-level hooks live in `hooks/hooks.json`. Plugin MCP config lives in `.mcp.json` at plugin root.

9. Required MCP integrations

MCP	Used by	Install method
context7	external-documentation-researcher, coding-agent	Plugin <code>.mcp.json</code> (HTTP)
github	pr-creator, pr-reviewer, backend/frontend bug finders, codebase-researcher	<code>/plugin install github@claude-plugins-official</code>
jira	requirements-product-analyst, implementation-planner	<code>/plugin install jira@claude-plugins-official</code> (when available) or <code>atlassian</code>
atlassian (Jira+Confluence)	requirements-product-analyst, implementation-planner	<code>/plugin install atlassian@claude-plugins-official</code>
linear, asana, notion	requirements-product-analyst, implementation-planner	<code>/plugin install <name>@claude-plugins-official</code>
playwright	frontend-bug-finder	Manual <code>/mcp add</code> or fork plugin's <code>.mcp.json</code>

chrome-devtools	frontend-bug-finder	Manual /mcp add
storybook, chromatic	frontend-bug-finder	Manual install (Storybook + Chromatic accounts required)
figma	frontend-bug-finder	/plugin install figma@claude-plugins-official
sentry (observability)	backend/frontend bug finders	/plugin install sentry@claude-plugins-official
serena, codanna, codegraphcontext, tree-sitter, codeql, srclight	codebase-researcher, coding-agent	Manual /mcp add per project's setup
database-readonly (per-project)	backend-bug-finder	Manual /mcp add; read-only credentials only

AC-9: .mcp.json documents which MCPs to install via plugin (recommended) and which require manual install. guard-mcp-mutations.sh blocks mutation-like calls unless CLAUDE_ALLOW_MCP_MUTATION=1.

10. Security & safety guardrails

ID	Guardrail	Enforcement
SG1	Database writes/migrations require explicit human approval	guard-git-and-dangerous-bash.sh, CLAUDE.md §8
SG2	No editing team-visible Claude config (repo/.claude/**, repo/CLAUDE.md, repo/.mcp.json)	guard-edit-path-policy.sh
SG3	No editing .env* files	Hook + Git hook
SG4	No force-push / --hard reset / clean -fd	guard-git-and-dangerous-bash.sh
SG5	MCP mutation calls blocked unless explicitly approved	guard-mcp-mutations.sh
SG6	Only orchestrator can spawn specialists; nested spawning blocked	guard-agent-spawn.sh
SG7	No Claude/AI attribution in commits or commit messages	Git commit-msg + pre-push hooks
SG8	No docs/ignored/**, env files, private artifacts in	Git pre-commit + pre-push hooks

SG9	commits/pushes Bug finders are read-only : cannot edit, commit, push, deploy, mutate prod data, change feature flags	disallowedTools: Edit, Write, NotebookEdit, Agent in agent frontmatter + finder-specific hard rules
-----	--	---

11. Acceptance test plan

Each test below is a smoke-level check; run after install to verify the system.

ID	Test	Pass condition
T1	/plugin install claude-code-orchestration@cc-orchestration	Plugin appears under /plugin Installed, no Errors tab entries
T2	/agents	Lists engineering-orchestrator + 18 specialists
T3	/hooks	Lists 5 hooks under claude-code-orchestration
T4	/mcp	Lists context7
T5	bash install_repo_bootstrap.sh --repo . --yes	Creates docs/ignored/{implementation,workbooks,context} /, adds excludes, configures core.hooksPath, drops wrapper scripts
T6	echo "Co-authored-by: Claude <noreply@anthropic.com>" > /tmp/msg && git commit --allow-empty -F /tmp/msg	Commit blocked by commit-msg Git hook
T7	mkdir -p docs/ignored/foo && echo x > docs/ignored/foo/bar.md && git add docs/ignored/foo/bar.md && git commit -m 'test'	Commit blocked by pre-commit Git hook
T8	Orchestrator asked to "implement feature X" without a plan	Returns pre-implementation response with Open Questions; does NOT write code
T9	Backend bug finder asked to investigate a stack trace	Returns the 9-section report format from §7.2; does not edit code

T10	Worktree created via new-worktree.sh foo ../repo-foo	repo-foo/docs/ignored/ exists; repo-foo/.env* copied; core.hooksPath configured in the new tree
-----	--	---

12. Migration from the legacy private overlay

The previous install_private_overlay.sh approach is **deprecated**. To migrate:

- Uninstall old user-level components:


```
rm -rf ~/.claude/agents/<old-slug>
rm -rf ~/.claude/skills/<old-slug>-*
rm -rf ~/.claude-overlays/<old-slug>
rm -f ~/.local/bin/claude-<old-slug>
```
- Install the new plugin per INSTALL.md Path A or B.
- Re-run install_repo_bootstrap.sh for each affected repo. It is idempotent and will refresh the Git hook templates.

Legacy files to delete from disk: claude_code_architecture_pack*.zip, claude_code_private_overlay*.zip, the old Claude_Code_Private_Overlay_FINAL_Guide.{pdf,docx}, and the install_private_overlay.sh script.

13. Open questions & known limitations

ID	Question / Limitation	Owner	Notes
OQ1	Per-agent permissionMode is dropped by Claude Code’s plugin loader. Do we need to add a session-startup check (“specialists must run in default permissionMode”) or is the hook layer enough?	Architecture	Currently the hook layer enforces the relevant blocks; permissionMode was redundant.
OQ2	The orchestrator’s tools: Agent(...) allowlist hard-codes 18 specialist names. If a user installs another plugin with	Architecture	Recommend: namespace-prefix the agent files (cco-engineering-orchestrator etc.) in a future v1.1 if

	overlapping subagent names, behavior is undefined.		conflicts surface.
OQ3	validation-matrix default rules are hard-coded for one specific stack (app/web, uv run pytest, schemas/queue-messages.json).	Per-repo	Each consuming repo must edit validation-matrix/SKILL.md or add a project-specific override.
OQ4	Agent teams are referenced but not configured (no team.json shipped).	Future	Add a worked example in a future minor release; the agent-team-decision skill currently only documents <i>when</i> to use them.
OQ5	The bootstrap configures core.hooksPath to a path under ~/.local/share/cc-orchestration/. If the user already has a tools like husky or pre-commit configured, the bootstrap warns instead of overriding. Manual merge required.	User	Documented in INSTALL.md §14.
OQ6	The frontend-bug-finder references Storybook + Chromatic MCPs, which require those services and are not in claude-plugins-official as of 2026-05.	User	Manual MCP install per-project.
OQ7	Workbook → long-lived doc promotion	Future	A worked example would help; not

	is documentation-refresh's responsibility but the conflict resolution between workbook timestamps and long-lived doc revisions is left to human review.		blocking.
OQ8	Branch / worktree wrappers currently assume origin/<default-branch> is the base. Forked-repo workflows (where upstream is the source of truth) need a flag.	Future	Add --base-remote flag to the wrappers.

14. Traceability matrix (requirement → implementation)

Req	Implementation file(s)
G1, A1–A19, HC1–HC8	plugins/claude-code-orchestration/agents/*.md
S1–S27	plugins/claude-code-orchestration/skills/*/SKILL.md
SG1, SG2, SG4	plugins/claude-code-orchestration/hooks/guard-git-and-dangerous-bash.sh, guard-edit-path-policy.sh
SG5	plugins/claude-code-orchestration/hooks/guard-mcp-mutations.sh
SG6	plugins/claude-code-orchestration/hooks/guard-agent-spawn.sh
HC1 (post-edit)	plugins/claude-code-orchestration/hooks/post-edit-policy-check.sh
SG7, SG8 (Git layer)	bootstrap/install_repo_bootstrap.sh (writes Git hook templates)
G3, G4, 6.4, 6.6	bootstrap/install_repo_bootstrap.sh (.git/info/exclude, .worktreeinclude, docs/ignored/, wrappers)
6.7	skills/agent-team-decision/SKILL.md
6.8	skills/root-cause-convergence/

D1	SKILL.md plugins/claude-code-orchestration/.claude-plugin/plugin.json
D2	marketplace/.claude-plugin/marketplace.json
D3	bootstrap/install_repo_bootstrap.sh
D4	docs/INSTALL.md (+ .docx)
Acceptance T1–T10	Manual smoke test post-install

v2 supplement

§0 (new) Authority of the custom spec over official plugins

Official plugins (feature-dev, superpowers, code-review, context7, microsoft-docs, serena, github, atlassian, playwright, chrome-devtools-mcp, claude-md-management, commit-commands, claude-code-setup, pr-review-toolkit, frontend-design) are **tools used by the custom agents**. They never replace custom agents. If a plugin’s workflow conflicts with a rule here, the rule here wins.

§4 (extension) New agents (v2)

ID	Agent	Purpose
A20	architecture-enforcer	Verifies code aligns with codebase architecture. Returns ALIGNED / DRIFT_ACCEPTABLE_PER_PLAN / DRIFT_REQUIRES_HUMAN_REVIEW / BLOCKED.
A21	data-architect	DB schema, migrations, indexes, query patterns, ORM mappings, data contracts. Read-only; never runs DDL/DML/migrations.
A22	comment-policy-checker	Pre-commit + pre-push gate. Scans code comments and commit messages for forbidden references (tickets, branches, plans, stages, phases, AI attribution). Returns CLEAN / BLOCK.
A23	parallel-research-	Orchestrates parallel

coordinator

codebase researchers + consistency review + gap-fill rounds + final synthesis. Up to 3 gap-fill rounds before escalating.

§5 (extension) New skills (v2)

ID	Skill	Purpose
S28	greenfield-vs-brownfield	Pick implementation mode; apply stricter brownfield rules by default.
S29	plan-review-cycle	3 parallel reviewers (requirements, architecture, feasibility) × N rounds → Plan-Final.md.
S30	parallel-codebase-research-cycle	Multi-round parallel research with cross-review and gap-fill.
S31	parallelization-decision	Force explicit human approval before any fan-out. Default SERIALIZE.
S32	plan-folder-organization	One folder per feature under docs/ignored/implementation/<feature-slug>/. Slug never contains ticket ID.
S33	pr-merge-conflict-wait	After PR opens, poll GH mergeability + checks.
S34	assumption-validation-tests	Pre/post-test comparison; complements superpowers TDD.
S35	official-docs-first	Routing rule: vendor-specific source FIRST, context7 generic fallback.
S36	gitignore-local-hygiene	.git/info/exclude patterns for personal-only files.

§6 (extension) New workflows

6.11 Greenfield vs. brownfield

Default: brownfield (stricter). Detected by inspecting whether touched files exist.
Brownfield drift requires: - “Drift: ” declaration in implementation plan - Human approval in writing - Module Context File update

Greenfield: use feature-dev@claude-plugins-official's code-explorer + code-architect for design proposals; architecture-enforcer records the chosen design as the Module Context File.

AC-6.11: greenfield-vs-brownfield/SKILL.md exists. CLAUDE.md §8 references it.

6.12 Plan review cycle (multi-round convergence)

Plans accumulate stale content across revisions. The cycle: - Round 1: 3 parallel reviewers (requirements / architecture-enforcer / feasibility) → individual review files. - Round 2: consolidator merges → dedup → severity-rank. - Round 3: planner revises to address blockers + majors → Plan-v(n+1).md. - Round 4: re-review. - Max 5 rounds. Only Plan-Final.md is implementable.

AC-6.12: plan-review-cycle/SKILL.md + architecture-enforcer/agent.md + new orchestrator workflow.

6.13 Parallel codebase research cycle

For research touching >3 modules. Decompose → ASK PARALLELIZATION → fan-out → consistency reviewer → gap-fill (max 3 rounds) → final synthesis. Each researcher writes its own timestamped file; final synthesis is the only artifact downstream agents read.

AC-6.13: parallel-research-coordinator/agent.md + parallel-codebase-research-cycle/SKILL.md.

6.14 Parallelization decision is opt-in

Default SERIALIZE. Before any fan-out: enumerate stages, declare dependency type (none/data-only/files-overlap/unknown), ASK the human. Record in Parallelization-<ts>.md.

AC-6.14: parallelization-decision/SKILL.md + CLAUDE.md §7 + orchestrator step 8.

6.15 Assumption-validation tests (pre/post)

Unless strictly sure, write a test that fails first. Implement. Re-run. The pre-test FAIL output and post-test PASS output are persisted as artifacts. Compare. Complements superpowers red-green-refactor TDD enforcement when installed.

AC-6.15: assumption-validation-tests/SKILL.md + CLAUDE.md §6.

6.16 PR merge-conflict wait

After pr-creator opens a PR, poll GH mergeable + mergeStateStatus + statusCheckRollup. CONFLICTING → STOP. BEHIND → recommend rebase. Failing required checks → report. PR is “submitted” only when MERGEABLE + CLEAN.

AC-6.16: pr-merge-conflict-wait/SKILL.md + pr-creator/agent.md updated to invoke it.

6.17 Official docs first (routing rule)

For any external behavior claim: pick the MOST-SPECIFIC source first. Microsoft → microsoft-docs. Atlassian → atlassian. GitHub → github. Browser → chrome-devtools-mcp + playwright. Generic → context7. No plugin available → vendor MCP / WebSearch. Cite the source. No claim from training memory alone.

AC-6.17: official-docs-first/SKILL.md + CLAUDE.md \$3.

6.18 Comment / commit / PR metadata policy (4-layer enforcement)

Forbidden in code comments AND commit messages AND PR bodies: - Ticket IDs (Jira-style [A-Z]{2,10}-\d+) - Branch names (feature/..., fix/..., etc.) - Implementation plan filenames (Plan*.md, Stage-*.md) - Phase / stage references - Workbook references / docs/ignored/ paths - Claude / Anthropic / AI / GPT / Copilot attribution - /

Enforced at: 1. post-edit-policy-check.sh hook (after-edit warning) 2. comment-policy-checker agent (pre-commit structured verdict) 3. pre-push-guardian agent (final pre-push gate) 4. Git pre-commit + commit-msg + pre-push hooks installed by bootstrap

Bypass with ALLOW_COMMENT_POLICY_BYPASS=1 per commit only (for legitimate string literals).

AC-6.18: comment-policy-checker/agent.md + updated post-edit-policy-check.sh + updated bootstrap hooks.

§9 (extension) MCP / plugin routing — v2

Plugins are preferred over raw MCPs when both exist. Routing by topic:

Topic	First	Then
Microsoft / Azure / .NET	microsoft-docs@claude-plugins-official	context7, WebSearch
Atlassian / Jira / Confluence	atlassian@claude-plugins-official	context7, WebSearch
GitHub API	github@claude-plugins-official	context7, WebSearch
Browser / DOM	chrome-devtools-mcp + playwright	MDN, context7
Any other library/framework	context7@claude-plugins-official	vendor MCP, WebSearch
Codebase	serena@claude-plugins-official	codegraphcontext, codeql, codanna, tree-sitter, srclight, Grep
Postman	Postman MCP (manual /mcp add)	context7, WebSearch
Google Cloud / AWS	Vendor MCP (manual)	context7, WebSearch

AC-9.v2: .mcp.json has empty mcpServers (deferred to plugins). _recommendedPlugins in plugin.json lists all 15. CLAUDE.md §3 and §4 enforce the routing.

§10 (extension) New guardrails

ID	Guardrail	Enforcement
SG10	No ticket/branch/plan references in code comments	post-edit-policy-check.sh hook + comment-policy-checker agent
SG11	No process metadata in commit messages	Git commit-msg hook + comment-policy-checker
SG12	No silent parallelization	parallelization-decision skill + orchestrator step gate
SG13	PR never “submitted” with unknown mergeability	pr-merge-conflict-wait skill
SG14	Architecture drift requires explicit declaration + human approval	architecture-enforcer verdict gates
SG15	All external claims require citation	official-docs-first skill + CLAUDE.md §3
SG16	Plan never implementable without going through review cycle	plan-review-cycle skill + orchestrator gate

§15 (new) Plan folder organization

docs/ignored/implementation/<feature-slug>/
 README.md
 Requirements.md
 Plan-v1.md, Plan-v2.md, ..., Plan-Final.md
 Review-1-{requirements,architecture,feasibility}-<ts>.md
 Review-Consolidated-<n>-<ts>.md
 Parallelization-<ts>.md
 Limitations.md
 EdgeCases.md
 Stages/Stage-<n>-<title>.md
 Progress.md
 PostMortem.md

Slug: kebab-case, NO ticket IDs, NO branch names, NO team-internal identifiers.

AC-15: plan-folder-organization/SKILL.md defines the layout. implementation-planner enforces it.

§16 (new) Mobile / remote coding

Three viable options: 1. Claude.ai mobile app (Max/Team plans expose a Claude Code surface) 2. SSH into a workstation that runs Claude Code (Termius / Blink / Tailscale SSH) 3. Tailscale VPN + workstation

In all three, the agents, skills, hooks, and plugins live on the workstation — the phone is the I/O device.

§17 (new) Local git hygiene

Personal patterns go in `.git/info/exclude`, NEVER `.gitignore`. Bootstrap auto-populates standard patterns. `gitignore-local-hygiene` skill is invoked whenever Claude or the human creates a personal-only file. The user's pain story (untracked DPRD-3543 files following a checkout) is the canonical example.

AC-17: `gitignore-local-hygiene/SKILL.md` + bootstrap populates `.git/info/exclude` + `CLAUDE.md` §17.

§18 (new) Plugin dependencies

This plugin **recommends** (does not auto-install — that's the user's choice) these 15 official plugins. The bootstrap's `--print-plugins` flag prints the install commands. `CLAUDE.md` §21 names each.

context7 code-review feature-dev superpowers github atlassian
playwright chrome-devtools-mcp serena claude-md-management
commit-commands claude-code-setup pr-review-toolkit microsoft-docs
frontend-design

Plus, manual MCP installs when needed: - Postman MCP - codegraphcontext, tree-sitter, codanna, codeql, srclight (no plugin yet) - Vendor MCPs (Google Cloud, AWS, Stripe, Twilio, etc.) as needed

AC-18: `_recommendedPlugins` in `plugin.json` + `INSTALL.md` §3.1 + bootstrap `--print-plugins` flag.

Open questions / known limitations (v2 additions)

ID	Question	Notes
OQ9	The strengthened comment-policy regex may produce false-positives on legitimate string literals containing ticket-ID-like patterns.	Bypass with <code>ALLOW_COMMENT_POLICY_BYPASS=1</code> . Could be refined to better distinguish comments from string literals.
OQ10	The orchestrator's tools: <code>Agent(...)</code> allowlist now has 22 entries. If new official plugins arrive with	Mitigation: namespace-prefix in v2.1 if conflicts emerge.

	overlapping subagent names, conflict resolution is undefined.	
OQ11	Plan review cycle's "max 5 rounds" is heuristic. Some plans may legitimately need more.	Acceptable to escalate; document the reasoning.
OQ12	parallel-research-coordinator's gap-fill bound of 3 rounds may be too tight for huge codebases.	Tunable; escalation path defined.
OQ13	pr-merge-conflict-wait's 60-second poll budget is a heuristic. Some PRs need longer GH compute.	Recommend retry after 60s if still UNKNOWN.
OQ14	assumption-validation-tests and superpowers TDD may produce redundant artifacts.	Acceptable; the artifacts complement each other for audit.

v3.2 supplement

This supplement closes the gap between this spec, the plugin's CLAUDE.md, and the user's external global engineering rules. Every item below is encoded in plugins/claude-code-orchestration/CLAUDE.md v3.2.0.

§0.2 (new) Verification status discipline

Every claim about external behavior (frameworks, libraries, SDKs, APIs, CLIs, cloud services) must be backed by an MCP, official documentation, or WebSearch citation per CLAUDE.md §3. When a claim cannot be verified through any of those, it **MUST** be explicitly labeled **UNVERIFIED** in: - The pre-implementation response (§7.1 – §10 in CLAUDE.md) - The post-implementation response (§7.5 below – §11 in CLAUDE.md, under Remaining Risks)

The human may choose to accept the residual risk and proceed; the unverified status is a transparency requirement, not a hard stop.

AC-0.2: CLAUDE.md §3 enforces the labeling rule. official-docs-first skill documents the verification procedure.

§0.3 (new) Reuse-first directive

When existing utilities, abstractions, or reusable components in the codebase fit the confirmed requirements, prefer reusing them over writing new ones. New helpers that

duplicate existing functionality must be justified explicitly in the implementation plan and approved by the human.

AC-0.3: CLAUDE.md §4 declares this as a positive directive. code-reviewer and architecture-enforcer check for duplication during the review pipeline.

§0.4 (new) Simplest-solution preference

When two designs both satisfy the confirmed requirements, pick the one with less code, fewer abstractions, and fewer moving parts. Complexity must be justified by a confirmed requirement, not by anticipated future needs.

AC-0.4: CLAUDE.md §13 declares this preference. code-reviewer and performance-reviewer flag over-engineering during the review pipeline.

§0.5 (new) Lint and type-check as universal loop gates

In the implementation feedback loop, after targeted tests and broader validation run, the test agent runs lint and type-check **if the repo provides them** (see CLAUDE.md §20 for project defaults). The agent reports which gates ran and which were not configured. Lint/type-check are no longer “project-specific only” — they are first-class loop gates whenever the repo wires them up.

AC-0.5: CLAUDE.md §5 step 8 requires this. validation-matrix skill enumerates the per-stack defaults. test-agent reports the gates ran.

§0.6 (new) Canonical per-dependency documentation (surfaced in narrative authority)

Previously encoded only in the documentation-refresh skill, now also surfaced in CLAUDE.md §15 so every agent sees it on every session. Rules:

- **Naming:** one dedicated Markdown file per external service / framework / library / SDK / API / database / platform, under docs/ignored/<CanonicalName>Documentation.md. Canonical, stable name, no spaces, no separators. Examples: StripeDocumentation.md, ReactDocumentation.md, PostgreSQLDocumentation.md, OpenAIApiDocumentation.md.
- **Check before create:** before creating a new documentation file, check whether the corresponding canonical file already exists. If it does, UPDATE it. Do not create duplicates under alternative names.
- **One dependency per file:** never mix multiple services / frameworks / SDKs in the same file unless explicitly requested.
- **Required contents** (verified, current, non-conflicting): what it is and why this repo uses it; verified versions / API versions / SDK versions / platform versions; official documentation or MCP sources used for verification; capabilities verified; constraints, limitations, gotchas, edge cases, and security requirements; repo integration points (file paths and modules); interfaces, endpoints, config keys, commands, or signatures actually used; architectural decisions and why they were made.

- **On change:** when new information is discovered or behavior/implementation changes, immediately update the corresponding file. Rewrite stale sections; do not append contradictory notes. Remove outdated, duplicate, or conflicting content. Keep each file internally consistent.
- **On removal:** if a dependency is removed or no longer relevant, update or delete its documentation so stale info doesn't linger.

AC-0.6: CLAUDE.md §15 surfaces these rules. documentation-refresh skill encodes the full procedure. documentation-reviewer agent invokes the skill after every approved unit.

§7.5 (new) Required post-implementation response per unit

```
## What Changed
## Files Affected
## Documentation Files Created / Updated
## Summary of Documentation Changes      (what actually changed in each file,
not just the file list)
## Tests / Lint / Type-Check Results      (label any gate that was not configured
for this repo)
## Pre/Post Test Comparison                (per §6.15)
## Architecture Alignment Verdict          (from architecture-enforcer)
## Comment Policy Verdict                  (from comment-policy-checker)
## Rule Compliance Self-Check              (note any rule from CLAUDE.md §1-§24
whose compliance is uncertain or partial; "fully compliant" if none)
## Remaining Risks / Open Issues           (include any UNVERIFIED claims from
§0.2)
```

This template replaces and extends the implicit post-implementation expectations that were previously distributed across CLAUDE.md §11, the implementation-feedback-loop skill, and the per-agent output contracts. The two new line items (Summary of Documentation Changes, Rule Compliance Self-Check) close the parity gap with the user's external rule #11.

AC-7.5: CLAUDE.md §11 enforces this template. engineering-orchestrator produces it after each approved unit. implementation-feedback-loop skill references it.

§10 (extension) New guardrails (v3.2)

ID	Guardrail	Enforcement
SG17	Unverified external-behavior claims must be labeled UNVERIFIED, not omitted	CLAUDE.md §3 + orchestrator pre-/post-implementation templates
SG18	New utility / abstraction requires justification when an existing one fits	CLAUDE.md §4 + code-reviewer + architecture-enforcer
SG19	Complexity must be justified by a confirmed requirement	CLAUDE.md §13 + code-reviewer + performance-reviewer

SG20	Lint and type-check run as universal loop gates when the repo provides them	CLAUDE.md §5 step 8 + test-agent + validation-matrix
SG21	One canonical <Name>Documentation.md per external dependency; check-before-create	CLAUDE.md §15 + documentation-refresh skill + documentation-reviewer
SG22	Post-implementation response includes Rule Compliance Self-Check	CLAUDE.md §11 + engineering-orchestrator

Open questions / known limitations (v3.2 additions)

ID	Question	Notes
OQ15	“Lint / type-check if the repo provides them” requires the agent to detect what’s wired up. Detection may be wrong on repos with non-standard tooling.	Acceptable; the agent reports “not configured” rather than guessing. Human can override.
OQ16	The Rule Compliance Self-Check is an orchestrator self-attestation; it is not independently audited by another agent.	Acceptable for v3.2; could escalate to a separate compliance-auditor agent in v3.3 if attestation quality drops.
OQ17	The UNVERIFIED label is a transparency mechanism; the human still chooses whether to proceed. Behavior built on UNVERIFIED claims may regress when the underlying API changes.	Acceptable; the labeling is the contract, not a fix.